A Project Report

On

"SIGN LANGUAGE DETECTION USING MACHINE LEARNING"

Submitted to

CHHATTISGARH SWAMI VIVEKANAND TECHNICAL UNIVERSITY

BHILAI

Bachelor of Technology

In

Computer Science & Engineering

specialization in

Artificial intelligence and machine learning

By

MITHLESH CHANDRAKAR

DIPANSHU R. KALESHWAR

RITESH BHOY

RAVI VERMA

Under the Guidance of

Mr. MANOJ KUMAR SINGH

Asst. Professor(CSE)

Department of Computer Science Engineering

Shri Shankaracharya Technical Campus, Junwani, Bhilai

DECLARATION

We solemnly declare the report of the project work entitled "Face Recognition Based Attendance System" is based on our own work carried **out** during my study under **Mr. Manoj Kumar Singh, Assistant Professor** CSE department of **SSTC,** Bhilai.

I assert that the statements made, and conclusion drawn are an outcome of the project work. I further declare that to the best of my knowledge and belief that the report does not contain any part of any work which has been submitted for the award of any other degree / diploma / certificate in this university or any other university.

Mithlesh Chandrakar
Roll No: 301410920057
Enrollment No: BK5451

Dipanshu R. Kaleshwar
Roll No: 303610920004
Enrollment No: BK2309

Ritesh Bhoy
Roll No: 303610920013
Enrollment No: BK2383

Ravi Verma
Roll No: 301410320044
Enrollment No: BK4345

CERTIFICATE

This is to certify the report of the project submitted is an outcome of the project work entitled:" **Sign language detection using machine learning**" carried out by **Mithlesh Chandrakar** bearing roll no: **301410920057, Dipanshu R. Kaleshwar bearing** roll no: **303610920004, Ritesh Bhoy** bearing roll no: **303610920013, Ravi Verma** bearing roll no: **301410320044** and carried out under my guidance and supervision for the award of Degree in Bachelor of Technology in Computer Science and Engineering (SSTC) of Chhattisgarh Swami Vivekanand Technical University, Bhilai(CG).

To the best of my knowledge the report:

- Embodies the work of the candidate himself/herself.
- Has duly been completed.
- Fulfils the requirements of Ordinance relating to the BE degree of the University.
- Is up to the desired standards for the purpose of which it is submitted.

Dr. Samta Gajhbiye
Professor
HOD CSE(Specialization)
(Signature of HOD with seal)

Mr. Manoj Kumar Singh
Asst. Professor(CSE)
(Signature of the Guide)

The project work mentioned above hereby recommended and forwarded for examination and evaluation.

Date:

CERTIFICATE BY THE EXAMINERS

This is to certify that the project work entitled "**Sign Language detection using machine learning**" Submitted by

Mithlesh Chandrakar	Roll no. 301410920057	Enrollment no. BK5451
Dipanshu R. Kaleshwar	Roll no. 303610920004	Enrollment no. BK2309
Ritesh Bhoy	Roll no. 303610920013	Enrollment no. BK2383
Ravi Verm	Roll no. 301410320044	Enrollment no. BK4345

"Has been examined by the undersigned as a part of the examination for the award of Bachelor of Engineering degree in Computer Science & Engineering of Chhattisgarh Swami Vivekanand Technical University, Bhiai.

Internal Examiner
Date :2/5/24

External Examiner
Date:24/5/24

ACKNOWLEDGEMENT

Working on the project was a great experience for us. There were moments of anxiety when we could not solve our problem for several days. But we enjoyed every bit of the process and are thankful to all the people associated with us during this period.

We convey our sincere thanks to our project guide **Mr. Manoj Kumar Singh,** Assistant Professor for providing tats all sorts of facilities. His support and guidance helped us to carry out the project.We owe a great department of his gratitude for his constant advice. support, cooperation and encouragement throughout the project.

We would like to express our deep gratitude to our respected **Mr. Manoj Kumar Singh** for her help and support. We also pay special thanks for his helpline solution and comments enriched by her experience.

We would also like to express our deep gratitude to our college management **Shri I.P. Mishra, Chairman (Shri Gangajali Education** Society, **Bhilai), Mrs. Jaya Mishra, President (Shri Gangajali Education Society**, **Bhilai), Dr. P.B. Deshmukh, Director, SSTC and Dr. Samta Gajbhiye** (**H.O.D. CSE Department (Specialization)** for providing an educational ambience. It will be our pleasure to acknowledge, utmost cooperation and valuable suggestions from time to time given by our staff members of' our department to whom we owe our entire computer knowledge and also we would like to thank all those persons who have directly or indirectly helped us by providing books and computer peripherals and other necessary amenities which helped in the development of this project.

MITHLESH CHANDRAKAR
DIPANSHU R. KALESHWAR
RITESH BHOY
RAVI VERMA

INDEX

CONTENTS

CH**APTER 1: INTRODUCTION**

1.1 Introduction… .....

1.2 Scope.....

1.3 Problem Statement… .....

CHAPTER 2 SYSTEM SPECIFICATION

2.1 System Requirement… .....

2.2 System Features .....

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture .....

3.2 Modules in the System….....

3.3 Use Case Diagram.....

3.4 Activity Diagram.....

CHAPTER 4: IMPLEMENTATION

4.1 Code Snippets .....

4.2 Screen Shots….....

CHAPTER 5: CONCLUSION

5.1 Conclusion… .....

5.2 Future Scope .....

REFERENCES.....

APPENDICES .....

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Some of the major problems faced by a person who are unable to speak is they cannot express their emotion as freely in this world. Utilize that voice recognition and voice search systems in smartphone(s).[1] Audio results cannot be retrieved. They are not able to utilize (Artificial Intelligence/personal Butler) like google assistance, or Apple's SIRI etc.[2] because all those apps are based on voice controlling.

There is a need for such platforms for such kind of people. American Sign Language (ASL) is a complete, complex language that employs signs made by moving the hands combined with facial expressions and postures of the body. It is the go-to language of many North Americans who are not able to talk and is one of various communication alternatives used by people who are deaf or hard-of-hearing.[3]

While sign language is very essential for deaf-mute people, to communicate both with normal people and with themselves, is still getting less attention from the normal people.[4] The importance of sign language has been tending to ignored, unless there are areas of concern with individuals who are deaf-mute. One of the solutions to talk with the deaf-mute people is by using the mechanisms of sign language.

Hand gesture is one of the methods used in sign language for non-verbal communication. It is most commonly used by deaf & dumb people who have hearing or talking disorders to communicate among themselves or with normal people.[3] Various sign language systems have been developed by many manufacturers around the world but they are neither flexible nor cost-effective for the end users.[6]

1.2 SCOPE

One of the solutions to communicate with the deaf-mute people is by using the services of sign language interpreter. But the usage of sign language interpreters could be

expensive.[3] Cost-effective solution is required so that the deaf-mute and normal people can communicate normally and easily. [3]

Our strategy involves implementing such an application which detects pre-defined American sign language (ASL) through hand gestures. For the detection of movement of gesture, we would use basic level of hardware component like camera and interfacing is required. Our application would be a comprehensive User-friendly Based system built on PyQt5 module.

Instead of using technology like gloves or kinect, we are trying to solve this problem using state of the art computer vision and machine learning algorithms. [6]

This application will comprise of two core module one is that simply detects the gesture and displays appropriate alphabet. The second is after a certain amount of interval period the scanned frame would be stored into buffer so that a string of character could be generated forming a meaningful word.

Additionally, an-addon facility for the user would be available where a user can build their own custom-based gesture for a special character like period (.) or any delimiter so that a user could form a whole bunch of sentences enhancing this into paragraph and likewise. Whatever the predicted outcome was, it would be stored into a .txt file.

1.3 PROBLEM STATEMENT

Given a hand gesture, implementing such an application which detects pre-defined American sign language (ASL) in a real time through hand gestures and providing facility for the user to be able to store the result of the character detected in a txt file, also allowing such users to build their customized gesture so that the problems faced by persons who aren’t able to talk vocally can be accommodated with technological assistance and the barrier of expressing can be overshadowed.

CHAPTER 2

SYSTEM SPECIFICATION

2.1 SYSTEM REQUIREMENT

2.1.1 HARDWARE REQUIREMENTS

- Intel Core i3 3rd gen processor or later.
- 512 MB disk space.
- 512 MB RAM.
- Any external or inbuild camera with minimum pixel resolution 200 x 200 (300ppi or 150lpi) 4-megapixel cameras and up.

2.1.2 SOFTWARE REQUIREMENTS

- Microsoft Windows XP or later / Ubuntu 12.0 LTS or later /MAC OS 10.1 or later.
- Python Interpreter (3.6).
- TensorFlow framework, Keras API.
- PyQT5, Tkinter module.
- Python OpenCV2, scipy, qimage2ndarray, winGuiAuto, pypiwin32, sys, keyboard, pyttsx3, pillow libraries.

2.2 SYSTEM FEATURES

- User-friendly based GUI built using industrial standard PyQT5.
- Real time American standard character detection based on gesture made by user.
- Customized gesture generation.
- Forming a stream of sentences based on the gesture made after a certain interval of time.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

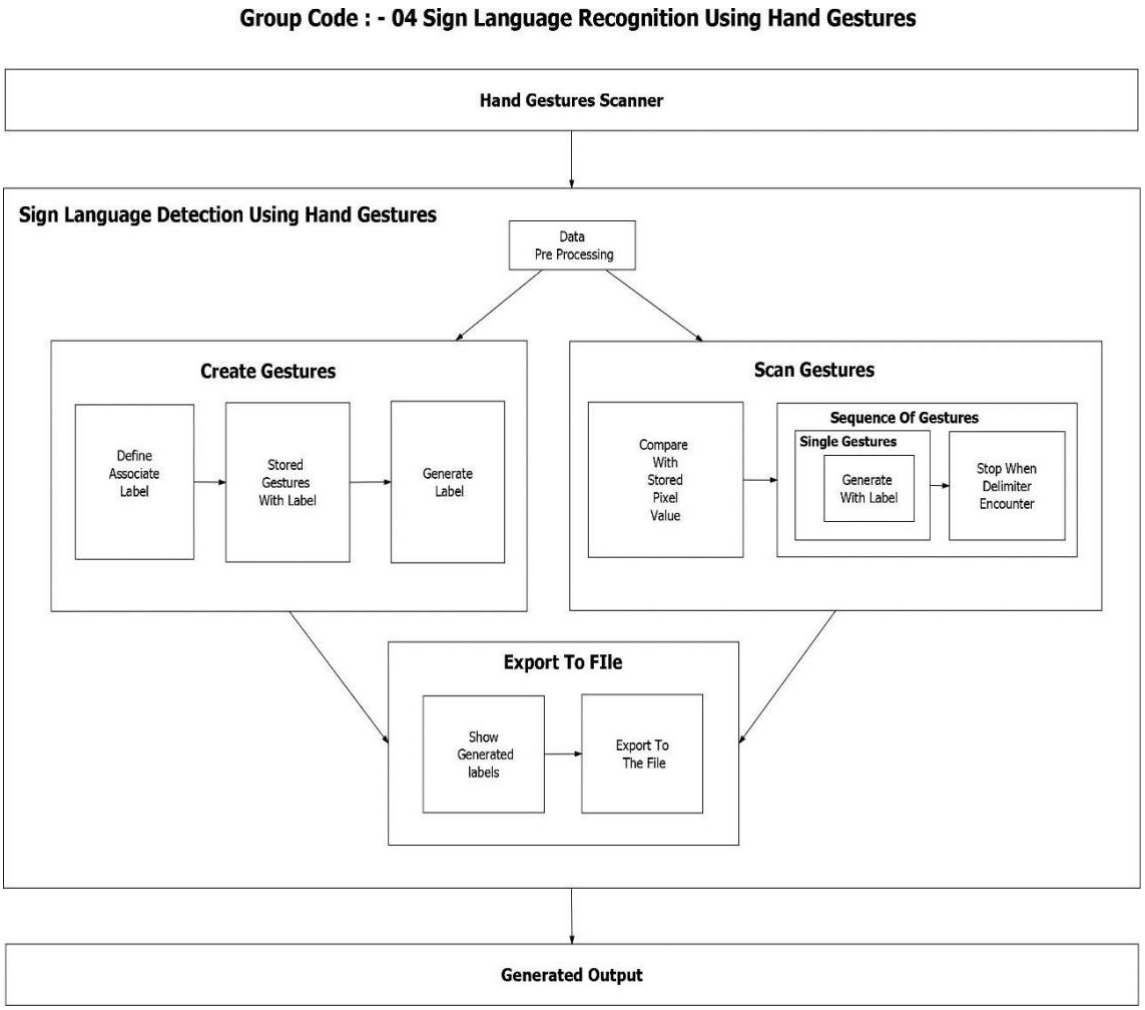

Fig 1: System Architecture for Sign Language Recognition Using Hand Gestures.

3.2 MODULES IN THE SYSTEM

- **Data Pre-Processing –** In this module, based on the object detected in front of the camera its binary images is being populated. Meaning the object will be filled with solid white and background will be filled with solid black. Based on the pixel's regions, their numerical value in range of either 0 or 1 isbeing given to next process for modules.

- **Scan Single Gesture –** A gesture scanner will be available in front of the end user where the user will have to do a hand gesture. Based on Pre-Processed module output, a user shall be able to see associated label assigned for each hand gestures, based on the predefined American Sign Language (ASL) standard inside the output window screen.
- **Create gesture –**A user will give a desired hand gesture as an input to the system with the text box available at the bottom of the screen where the user needs to type whatever he/she desires to associate that gesture with. This customize gesture will then be stored for future purposes and will be detected in the upcoming time.
- **Formation of a sentence –** A user will be able to select a delimiter and until that delimiter is encountered every scanned gesture character will be appended with the previous results forming a stream of meaning-full words and sentences.
- **Exporting –** A user would be able to export the results of the scanned character into an ASCII standard textual file format.

3.3 USE CASE DIAGRAM

Group Code: -4 Sign Language Recognition Using Hand Gestures

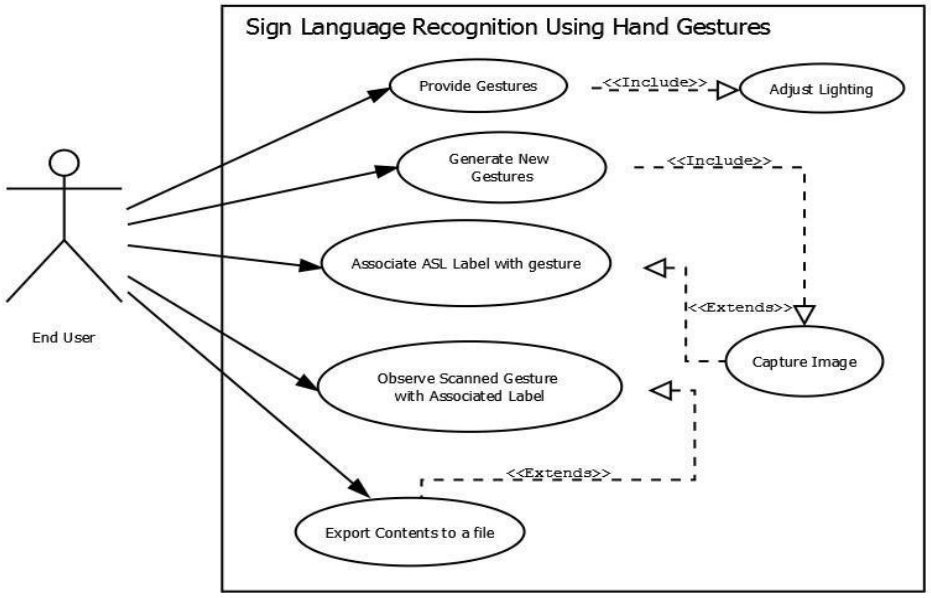

Fig 2: Use Case Diagram for Sign Language Recognition Using Hand Gestures.

3.4 ACTIVITY DIAGRAM

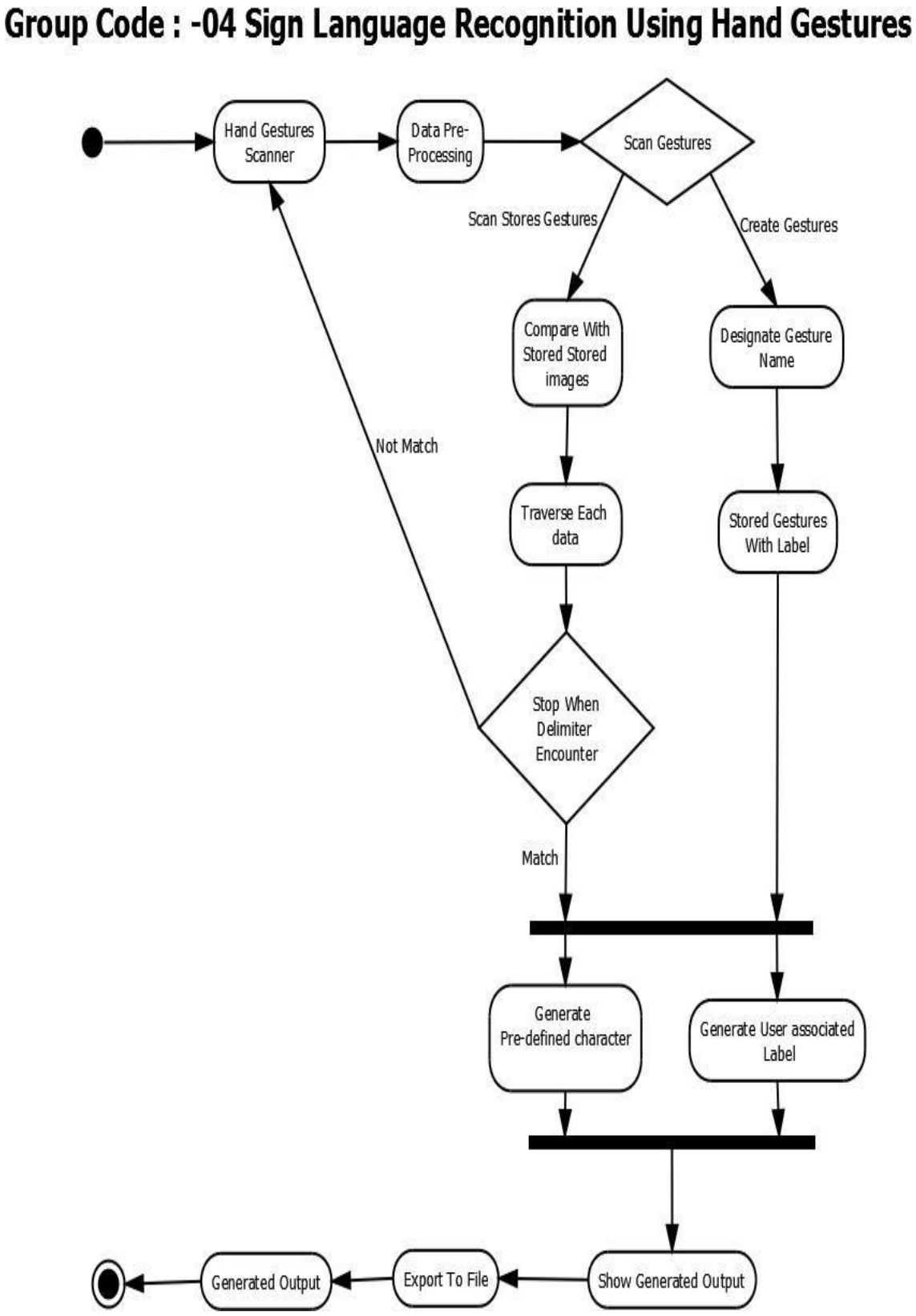

Fig 3: Activity Diagram for Sign Language Recognition Using Hand Gestures.

CHAPTER 4

IMPLEMENTATION

4.1 CODE SNIPPETS

Dashboard.py

```
__author___= 'Shadab Shaikh, Obaid Kazi, Ansari Mohd Adnan'

from PyQt5 import QtWidgets, uic
from PyQt5.QtWidgets import QMessageBox
from PyQt5.QtCore import QUrl
from PyQt5.QtGui import QImage
from PyQt5.QtGui import QPixmap
from PyQt5 import QtCore                                #importing pyqt5 libraries
from scipy.ndimage import imread                        #will help in reading the images
from PyQt5.QtCore import QTimer,Qt
from PyQt5 import QtGui
from tkinter import filedialog                          #for file export module
from tkinter import *
import tkinter as tk
from matplotlib import pyplot as plt                    #for gesture viewer
from matplotlib.widgets import Button
import sys                                              #for pyqt
import os                                               #for removal of files
import cv2                                              #for the camera operations
import numpy as np                                      #proceesing on images
import qimage2ndarray                                   #convers images into matrix
import win32api
import winGuiAuto
import win32gui
import win32co                          #for removing title cv2 window and always on top
import keyboard                         #for pressing keys
import pyttsx3                          #for tts assistance
import shutil                           #for removal of directories
index = 0                               #index used for gesture viewer
engine = pyttsx3.init()                 #engine initialization for audio tts assistance

def nothing(x):
        pass

image_x, image_y = 64,64                                #image resolution

from keras.models import load_model
classifier = load_model('ASLModel.h5')                  #loading the model

def fileSearch():
        """Searches each file ending with .png in SampleGestures dirrectory so that
        custom gesture could be passed to predictor() function"""
        fileEntry=[]
        for file in os.listdir("SampleGestures"):
            if file.endswith(".png"):
                fileEntry.append(file)
        return fileEntry

def load_images_from_folder(folder):
    """Searches each images in a specified directory"""
    images = []
    for filename in os.listdir(folder):
        img = cv2.imread(os.path.join(folder,filename))
        if img is not None:
            images.append(img)
    return images

def toggle_imagesfwd(event):
        """displays next images act as a gesutre viewer"""
        img=load_images_from_folder('TempGest/')
        global index

        index += 1

        try:
                if index < len(img):
                        plt.axes()
                        plt.imshow(img[index])
                        plt.draw()
```

```
        except:
                pass

def toggle_imagesrev(event):
        """displays previous images act as a gesutre viewer"""
        img=load_images_from_folder('TempGest/')
        global index

        index -= 1

        try:
                if index < len(img) and index>=0:
                        plt.axes()
                        plt.imshow(img[index])
                        plt.draw()
        except:

                pass

def openimg():
        """displays predefined gesture images at right most window"""
        cv2.namedWindow("Image", cv2.WINDOW_NORMAL )
        image = cv2.imread('template.png')
        cv2.imshow("Image",image)
        cv2.setWindowProperty("Image",cv2.WND_PROP_FULLSCREEN,cv2.WINDOW_FULLSCREEN)
        cv2.resizeWindow("Image",298,430)
        cv2.moveWindow("Image", 1052,214)


def removeFile():
        """Removes the temp.txt and tempgest directory if any stop button is pressed oor
        application is closed"""
        try:
                os.remove("temp.txt")
        except:
                pass
        try:
                shutil.rmtree("TempGest")
        except:
                pass


def clearfunc(cam):
        """shut downs the opened camera and calls removeFile() Func"""
        cam.release()
        cv2.destroyAllWindows()
        removeFile()

def clearfunc2(cam):
        """shut downs the opened camera"""
        cam.release()
        cv2.destroyAllWindows()

def saveBuff(self,cam,finalBuffer):
        """Save the file as temp.txt if save button is pressed in sentence formation
        through gui"""
        cam.release()
        cv2.destroyAllWindows()
        if(len(finalBuffer)>=1):
                f=open("temp.txt","w")
                for i in finalBuffer:
                        f.write(i)
                f.close()


def capture_images(self,cam,saveimg,mask):
        """Saves the images for custom gestures if button is pressed in custom gesture
        generationn through gui"""
        cam.release()
        cv2.destroyAllWindows()
        if not os.path.exists('./SampleGestures'):
                os.mkdir('./SampleGestures')

        gesname=saveimg[-1]
        if(len(gesname)>=1):
                img_name = "./SampleGestures/"+"{}.png".format(str(gesname))
                save_img = cv2.resize(mask, (image_x, image_y))
                cv2.imwrite(img_name, save_img)
```

```
def controlTimer(self):
        # if timer is stopped
        self.timer.isActive()
        # create video capture
        self.cam = cv2.VideoCapture(0)
        # start timer
        self.timer.start(20)


def predictor():
        """ Depending on model loaded and customgesture saved prediction is made by
        checking array or through SiFt algo"""
        import numpy as np
        from keras.preprocessing import image
        test_image = image.load_img('1.png', target_size=(64, 64))
        test_image = image.img_to_array(test_image)
        test_image = np.expand_dims(test_image, axis = 0)
        result = classifier.predict(test_image)
        gesname=''
        fileEntry=fileSearch()
        for i in range(len(fileEntry)):
                image_to_compare = cv2.imread("./SampleGestures/"+fileEntry[i])
                original = cv2.imread("1.png")
                sift = cv2.xfeatures2d.SIFT_create()
                kp_1, desc_1 = sift.detectAndCompute(original, None)
                kp_2, desc_2 = sift.detectAndCompute(image_to_compare, None)

                index_params = dict(algorithm=0, trees=5)
                search_params = dict()
                flann = cv2.FlannBasedMatcher(index_params, search_params)

                matches = flann.knnMatch(desc_1, desc_2, k=2)

                good_points = []
                ratio = 0.6
                for m, n in matches:
                      if m.distance < ratio*n.distance:
                            good_points.append(m)
                if(abs(len(good_points)+len(matches))>20):
                        #goodpoints and matcches sum from 1.png and customgestureimages
                        #is grater than 20
                        gesname=fileEntry[i]
                        gesname=gesname.replace('.png','')
                        if(gesname=='sp'):
                                #sp is replaced with <space>
                                gesname=' '
                        return gesname

        if result[0][0] == 1:
                  return 'A'
        elif result[0][1] == 1:
                  return 'B'
        elif result[0][2] == 1:
                  return 'C'
        elif result[0][3] == 1:
                  return 'D'
        elif result[0][4] == 1:
                  return 'E'
        elif result[0][5] == 1:
                  return 'F'
        elif result[0][6] == 1:
                  return 'G'
        elif result[0][7] == 1:
                  return 'H'
        elif result[0][8] == 1:
                  return 'I'
        elif result[0][9] == 1:
                  return 'J'
        elif result[0][10] == 1:
                  return 'K'
        elif result[0][11] == 1:
                  return 'L'
        elif result[0][12] == 1:
                  return 'M'
        elif result[0][13] == 1:
                  return 'N'
```

```
        elif result[0][14] == 1:
                return 'O'
        elif result[0][15] == 1:
                return 'P'
        elif result[0][16] == 1:
                return 'Q'
        elif result[0][17] == 1:
                return 'R'
        elif result[0][18] == 1:
                return 'S'
        elif result[0][19] == 1:
                return 'T'
        elif result[0][20] == 1:
                return 'U'
        elif result[0][21] == 1:
                return 'V'
        elif result[0][22] == 1:
                return 'W'
        elif result[0][23] == 1:
                return 'X'
        elif result[0][24] == 1:
                return 'Y'
        elif result[0][25] == 1:
                return 'Z'


def checkFile():
        """retrieve the content of temp.txt for export module """
        checkfile=os.path.isfile('temp.txt')
        if(checkfile==True):
                fr=open("temp.txt","r")
                content=fr.read()
                fr.close()
        else:
                content="No Content Available"
        return content

class Dashboard(QtWidgets.QMainWindow):


        def___init__(self):
                super(Dashboard, self)._init _()
                self.setWindowFlags(QtCore.Qt.WindowCloseButtonHint |
                QtCore.Qt.WindowMinimizeButtonHint | QtCore.Qt.FramelessWindowHint)
                cap = cv2.VideoCapture('gestfinal2.min.mp4')

                # Read until video is completed
                while(cap.isOpened()):
                        ret, frame = cap.read()
                        if ret == True:
                # Capture frame-by-frame
                                ret, frame = cap.read()
                                cv2.namedWindow("mask", cv2.WINDOW_NORMAL)
                                cv2.imshow("mask", frame)

                                cv2.setWindowProperty("mask",cv2.WND_PROP_FULLSCREEN,cv2.W
                                INDOW_FULLSCREEN)
                                cv2.resizeWindow("mask",720,400)
                                cv2.moveWindow("mask", 320,220)

                                if cv2.waitKey(25) & 0xFF == ord('q'):
                                        break

                        else:
                                break

                # When everything done, release
                cap.release()

                # Closes all the frames
                cv2.destroyAllWindows()
                self.setWindowIcon(QtGui.QIcon('icons/windowLogo.png'))
                self.title = 'Sign language Recognition'
                uic.loadUi('UI_Files/dash.ui', self)
                self.setWindowTitle(self.title)
                self.timer = QTimer()
                self.create.clicked.connect(self.createGest)
```

```
        self.exp2.clicked.connect(self.exportFile)
        self.scan_sen.clicked.connect(self.scanSent)
        if(self.scan_sinlge.clicked.connect(self.scanSingle)==True):
                self.timer.timeout.connect(self.scanSingle)
        self.create.setCursor(QtGui.QCursor(QtCore.Qt.PointingHandCursor))
        self.scan_sen.setCursor(QtGui.QCursor(QtCore.Qt.PointingHandCursor))
        self.scan_sinlge.setCursor(QtGui.QCursor(QtCore.Qt.PointingHandCursor))
        self.exp2.setCursor(QtGui.QCursor(QtCore.Qt.PointingHandCursor))
        self.exit_button.clicked.connect(self.quitApplication)
        self._layout = self.layout()
        self.label_3 = QtWidgets.QLabel()
        movie = QtGui.QMovie("icons/dashAnimation.gif")
        self.label_3.setMovie(movie)
        self.label_3.setGeometry(0,160,780,441)
        movie.start()
        self._layout.addWidget(self.label_3)
        self.setObjectName('Message_Window')

def quitApplication(self):


        """shutsdown the GUI window along with removal of files"""
        userReply = QMessageBox.question(self, 'Quit Application', "Are you sure
        you want to quit this app?", QMessageBox.Yes | QMessageBox.No,
        QMessageBox.No)
        if userReply == QMessageBox.Yes:
                removeFile()
                keyboard.press_and_release('alt+F4')

def createGest(self):


        """ Custom gesture generation module"""
        try:
                clearfunc(self.cam)
        except:
                pass
        gesname=""
        uic.loadUi('UI_Files/create_gest.ui', self)
        self.setWindowTitle(self.title)
        self.create.clicked.connect(self.createGest)
        self.exp2.clicked.connect(self.exportFile)
        if(self.scan_sen.clicked.connect(self.scanSent)):
                controlTimer(self)
        self.scan_sinlge.clicked.connect(self.scanSingle)
        self.linkButton.clicked.connect(openimg)
        self.create.setCursor(QtGui.QCursor(QtCore.Qt.PointingHandCursor))
        self.scan_sen.setCursor(QtGui.QCursor(QtCore.Qt.PointingHandCursor))
        self.scan_sinlge.setCursor(QtGui.QCursor(QtCore.Qt.PointingHandCursor))
        self.exp2.setCursor(QtGui.QCursor(QtCore.Qt.PointingHandCursor))
        self.pushButton_2.clicked.connect(lambda:clearfunc(self.cam))
        try:
                self.exit_button.clicked.connect(lambda:clearfunc(self.cam))
        except:
                pass
        self.exit_button.clicked.connect(self.quitApplication)
        self.plainTextEdit.setPlaceholderText("Enter Gesture Name Here")
        img_text = ''
        saveimg=[]
        while True:
                ret, frame = self.cam.read()
                frame = cv2.flip(frame,1)
                try:
                        frame=cv2.resize(frame,(321,270))
                        frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
                        img2 = cv2.rectangle(frame, (150,50),(300,200), (0,255,0),
                        thickness=2, lineType=8, shift=0)
                except:
                        keyboard.press_and_release('esc')

                height2, width2, channel2 = img2.shape
                step2 = channel2 * width2
                # create QImage from image
                qImg2 = QImage(img2.data, width2, height2, step2,
                QImage.Format_RGB888)
                # show image in img_label
                try:
```

```
                        self.label_3.setPixmap(QPixmap.fromImage(qImg2))
                        slider2=self.trackbar.value()
                except:
                        pass

                lower_blue = np.array([0, 0, 0])
                upper_blue = np.array([179, 255, slider2])
                imcrop = img2[52:198, 152:298]
                hsv = cv2.cvtColor(imcrop, cv2.COLOR_BGR2HSV)
                mask = cv2.inRange(hsv, lower_blue, upper_blue)

                cv2.namedWindow("mask", cv2.WINDOW_NORMAL )
                cv2.imshow("mask", mask)

                cv2.setWindowProperty("mask",cv2.WND_PROP_FULLSCREEN,cv2.WINDOW_F
                ULLSCREEN)
                cv2.resizeWindow("mask",170,160)
                cv2.moveWindow("mask", 766,271)

                hwnd = winGuiAuto.findTopWindow("mask")
                win32gui.SetWindowPos(hwnd, win32con.HWND_TOP,
                0,0,0,0,win32con.SWP_NOMOVE | win32con.SWP_NOSIZE |
                win32con.SWP_NOACTIVATE)

                try:
                        ges_name = self.plainTextEdit.toPlainText()
                except:
                        pass
                if(len(ges_name)>=1):
                        saveimg.append(ges_name)
                else:
                        saveimg.append(ges_name)
                        ges_name=''

                try:
                        self.pushButton.clicked.connect(lambda:capture_images(self
                        ,self.cam,saveimg,mask))
                except:
                        pass


                gesname=saveimg[-1]

                if keyboard.is_pressed('shift+s'):
                        if not os.path.exists('./SampleGestures'):
                                os.mkdir('./SampleGestures')
                        if(len(gesname)>=1):
                                img_name =
                                "./SampleGestures/"+"{}.png".format(str(gesname))
                                save_img = cv2.resize(mask, (image_x, image_y))
                                cv2.imwrite(img_name, save_img)
                        break

                if cv2.waitKey(1) == 27:
                        break

        self.cam.release()
        cv2.destroyAllWindows()

        if os.path.exists("./SampleGestures/"+str(gesname)+".png"):
                QtWidgets.QMessageBox.about(self, "Success", "Gesture Saved
                Successfully!")

def exportFile(self):


        """export file module with tts assistance and gesturre viewer"""
        try:
                clearfunc2(self.cam)
        except:
                pass
        uic.loadUi('UI_Files/export.ui', self)
        self.setWindowTitle(self.title)
        self.create.clicked.connect(self.createGest)
        self.exp2.clicked.connect(self.exportFile)
        self.scan_sen.clicked.connect(self.scanSent)
        self.scan_sinlge.clicked.connect(self.scanSingle)
        self.create.setCursor(QtGui.QCursor(QtCore.Qt.PointingHandCursor))
```

```
        self.scan_sen.setCursor(QtGui.QCursor(QtCore.Qt.PointingHandCursor))
        self.scan_sinlge.setCursor(QtGui.QCursor(QtCore.Qt.PointingHandCursor))
        self.exp2.setCursor(QtGui.QCursor(QtCore.Qt.PointingHandCursor))
        self.exit_button.clicked.connect(self.quitApplication)
        content=checkFile()
        self.textBrowser_98.setText("          "+content)
        engine.say(str(content).lower())
        try:
                engine.runAndWait()
        except:
                pass
        if(content=="File Not Found"):
                self.pushButton_2.setEnabled(False)
                self.pushButton_3.setEnabled(False)
        else:
                self.pushButton_2.clicked.connect(self.on_click)
                try:
                        self.pushButton_3.clicked.connect(self.gestureViewer)
                except:
                        pass

def on_click(self):




        """Opens tkinter window to save file at desired location """
        content=checkFile()
        root=Tk()
        root.withdraw()
        root.filename = filedialog.asksaveasfilename(initialdir = "/",title =
        "Select file",filetypes = (("Text files","*.txt"),("all files","*.*")))
        name=root.filename
        #fr.close()
        fw=open(name+".txt","w")
        if(content=='No Content Available'):
                content=" "
        fw.write(content)
        try:
                os.remove("temp.txt")
                shutil.rmtree("TempGest")
        except:
                QtWidgets.QMessageBox.about(self, "Information", "Nothing to
                export")
        fw.close()
        root.destroy()

        if not os.path.exists('temp.txt'):
                if os.path.exists('.txt'):
                        os.remove('.txt')
                else:
                        QtWidgets.QMessageBox.about(self, "Information", "File
                        saved successfully!")
                        self.textBrowser_98.setText("          ")

def gestureViewer(self):


        """gesture viewer through matplotlib """
        try:
                img=load_images_from_folder('TempGest/')
                plt.imshow(img[index])
        except:
                plt.text(0.5, 0.5, 'No new Gesture Available',
        horizontalalignment='center',verticalalignment='center')
        axcut = plt.axes([0.9, 0.0, 0.1, 0.075])
        axcut1 = plt.axes([0.0, 0.0, 0.1, 0.075])
        bcut = Button(axcut, 'Next', color='dodgerblue', hovercolor='lightgreen')
        bcut1 = Button(axcut1, 'Previous', color='dodgerblue',
        hovercolor='lightgreen')

        #plt.connect('button_press_event', toggle_imagesfwd)
        bcut.on_clicked(toggle_imagesfwd)
        bcut1.on_clicked(toggle_imagesrev)
        plt.show()
        axcut._button = bcut          #creating a reference for that element
        axcut1._button1 = bcut1
```

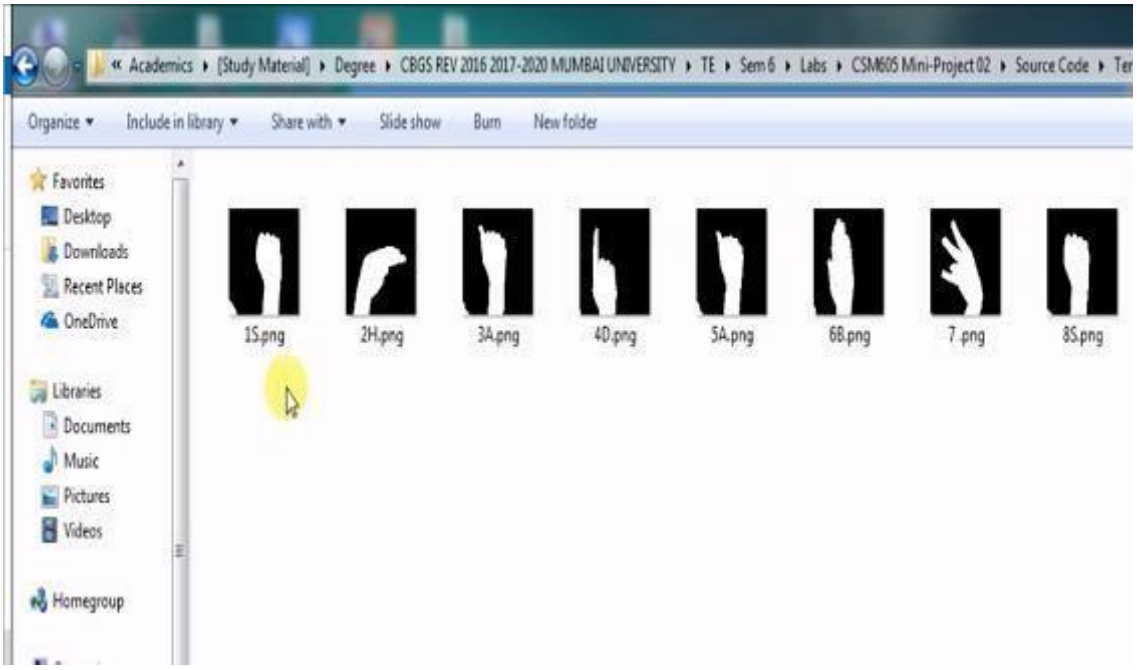

Fig 4.2.15: Content of Newly Generated TempGest Directory.

Fig 4.2.16: Export with TTS Assistance and Gesture Viewer.

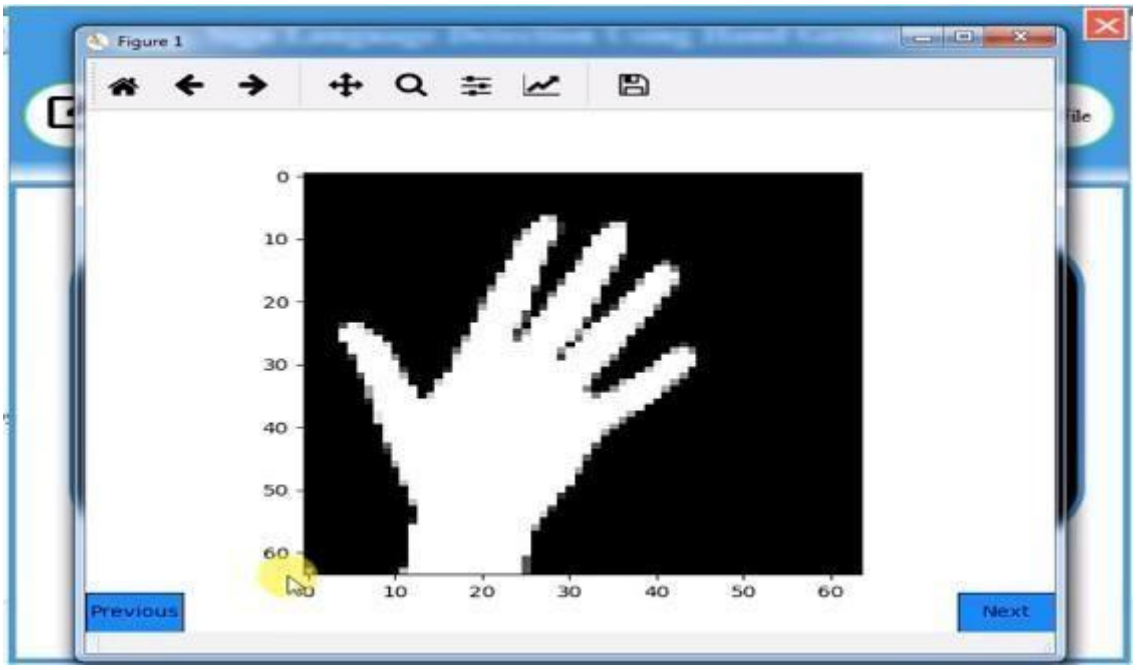

Fig 4.2.17: Gesture Viewer Sample.

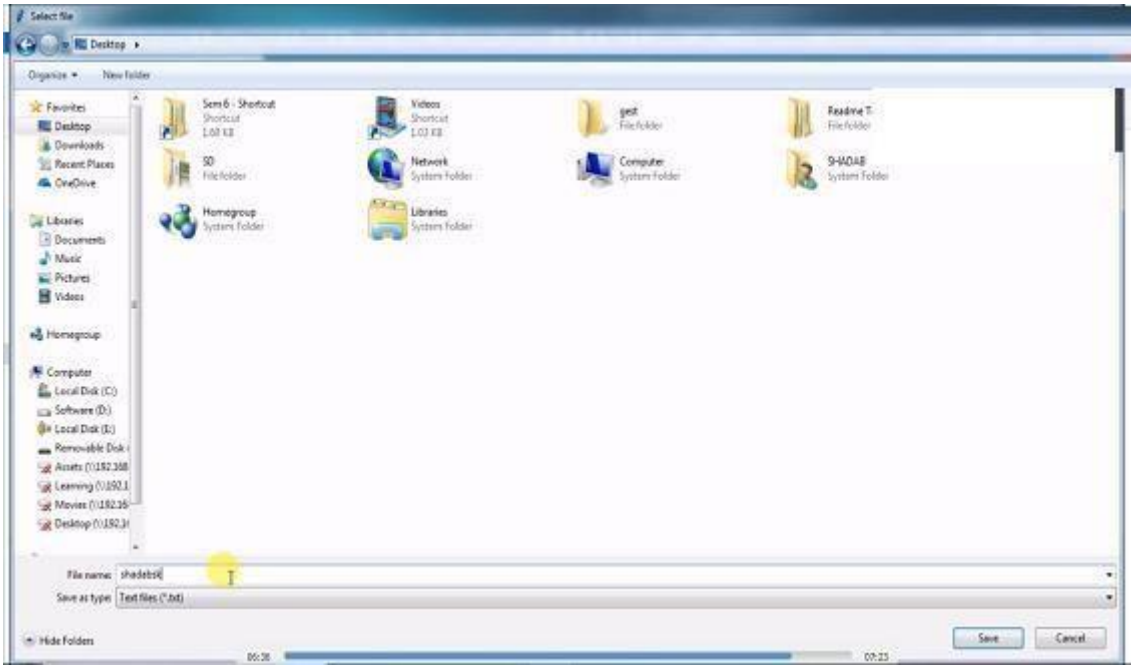

Fig 4.2.18: Exporting the File at Desired Location.

Fig 4.2.19: File Saved Successfully.

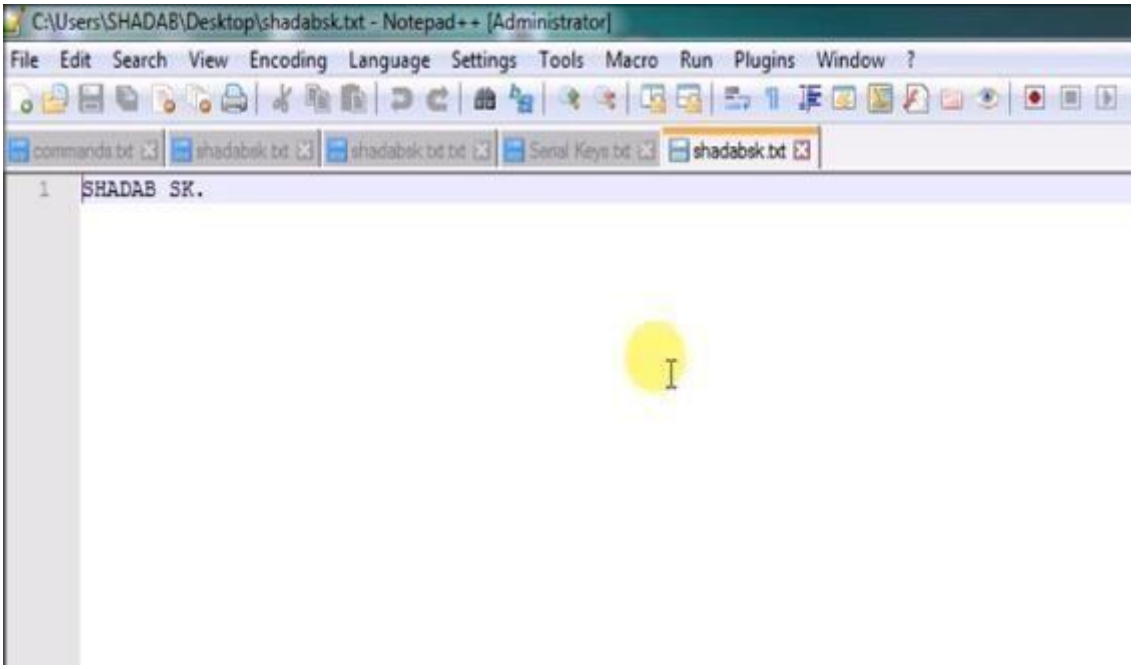

Fig 4.2.20: Content of Newly Saved File.

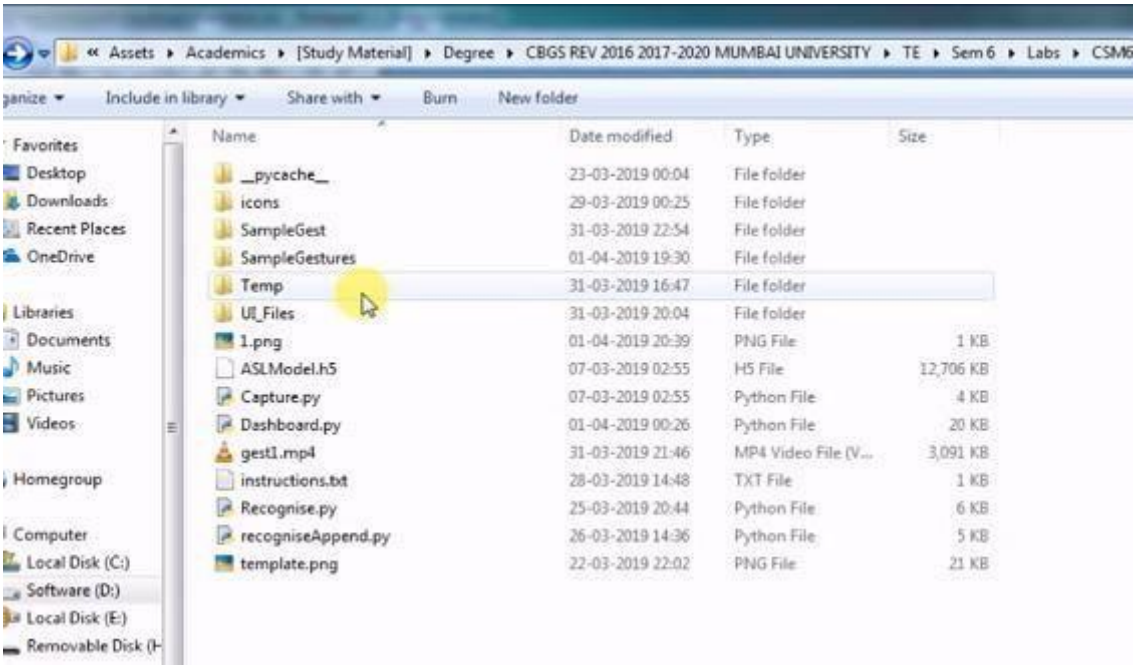

Fig 4.2.21: Temporary Files Deleted.

Fig 4.2.22: No Content Available.

Fig 4.2.23: Quitting the Application.

CHAPTER 5

CONCLUSION

5.1 CONCLUSION

From this project/application we have tried to overshadow some of the major problems faced by the disabled persons in terms of talking. We found out the root cause of why they can't express more freely. The result that we got was the other side of the audience are not able to interpret what these persons are trying to say or what is the message that they want to convey.

Thereby this application serves the person who wants to learn and talk in sign languages. With this application a person will quickly adapt various gestures and their meaning as per ASL standards. They can quickly learn what alphabet is assigned to which gesture. Add-on to this custom gesture facility is also provided along with sentence formation. A user need not be a literate person if they know the action of the gesture, they can quickly form the gesture and appropriate assigned character will be shown onto the screen.

Concerning to the implementation, we have used TensorFlow framework, with keras API. And for the user feasibility complete front-end is designed using PyQT5. Appropriate user-friendly messages are prompted as per the user actions along with what gesture means which character window. Additionally, an export to file module is also provided with TTS(Text-To-Speech) assistance meaning whatever the sentence was formed a user will be able to listen to it and then quickly export along with observing what gesture he/she made during the sentence formation.

5.2 FUTURE SCOPE

- It can be integrated with various search engines and texting application such as google, WhatsApp. So that even the illiterate people could be able to chat with other persons, or query something from web just with the help of gesture.
- This project is working on image currently, further development can lead to detecting the motion of video sequence and assigning it to a meaningful sentence with TTS assistance.

REFERENCES

REFERENCES

[1] Shobhit Agarwal, “What are some problems faced by deaf and dumb people while using todays common tech like phones and PCs”, 2017 [Online]. Available: https://www.quora.com/What-are-some-problems-faced-by-deaf-and-dumb-people-while-using-todays-common-tech-like-phones-and-PCs, [Accessed April 06, 2019].

[2] NIDCD, “american sign language”, 2017 [Online]. Available: https://www.nidcd.nih.gov/health/american-sign-language, [Accessed April 06, 2019].

[3] Suharjito MT, “Sign Language Recognition Application Systems for Deaf-Mute People A Review Based on Input-Process-Output”, 2017 [Online]. Available: https://www.academia.edu/35314119/Sign_Language_Recognition_Application_Systems_for_Deaf-Mute_People_A_Review_Based_on_Input-Process-Output [Accessed April 06, 2019].

[4] M. Ibrahim, “Sign Language Translation via Image Processing”, [Online]. Available: https://www.kics.edu.pk/project/startup/203 [Accessed April 06, 2019].

[5] NAD, “American sign language-community and culture frequently asked questions”, 2017 [Online]. Available: https://www.nad.org/resources/american-sign-language/community-and-culture-frequently-asked-questions/ [Accessed April 06, 2019].

[6] Sanil Jain and K.V.Sameer Raja, “Indian Sign Language Character Recognition” , [Online]. Available: https://cse.iitk.ac.in/users/cs365/2015/_submissions/vinsam/report.pdf [Accessed April 06, 2019].